

Data Science Assessment – Task 1 Report

Overview

This document addresses **Task 1** of the Data Science assessment.

1. *Time series challenge*: Predict the weekly number of expected service requests per hex that will be created each week using sr_hex.csv, for 4 weeks past the end of the dataset.
-

Data Preparation and Preprocessing

Python code: [DataScienceTask1_Preprocessing.py](#)

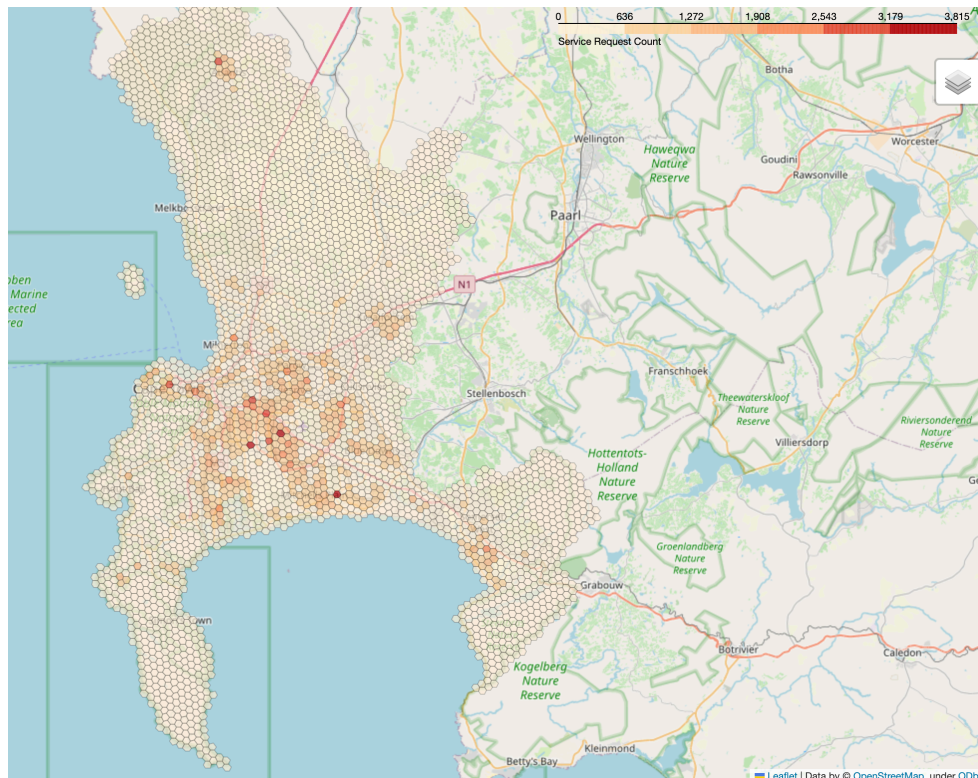
- This script loads the raw event records CSV.
- This script also attributes the geographical hex data to the count dataframe.
Note: this attribution is not used, as the sr_hex.csv.gz dataset with attribution already completed is used hence.
- Each record includes:
 - a timestamp
 - a hex_index identifier.

Preprocessing Workflow:

1. **Convert Timestamps**
 - creation_timestamp is converted to datetime format.
2. **Aggregate by Week**
 - Each timestamp is mapped to the start of its week (week_start).
 - Counts of events per hex per week are calculated.
3. **Pivot to Wide Format**
 - Data is reshaped so:
 - Rows = unique hex_index
 - Columns = counts for each week.
 - Missing weeks are filled with zeros.

Output:

- A spatial / temporal pivoted DataFrame (pivoted) used in subsequent modelling steps.
- Visualizations are generated to show spatial distributions and counts across hexes.



Prediction Objective

Goal:

Predict the event counts for each hex **four weeks into the future**.

Train/Test Split:

- **Training Set:** 80% of hexes
- **Test Set:** 20% of hexes

Modelling Approaches

We trained several models to forecast counts:

1. SARIMAX Model (*SARIMAX_prediction.py*)

Approach:

- Each hex treated independently.
- **Input:**
 - For each hex, historical weekly counts excluding the last 4 weeks.
 - Example: if 53 weeks total, the first 49 weeks are training.
- **No exogenous regressors included.**

Modelling Details:

- A separate SARIMAX model is fitted to each hex time series.
- Counts are modelled as a linear function of past observations.

2. Random Forest Regressor (*RandomForestRegression.py*)

Approach:

- Single model trained across all hexes.

Input:

- For each hex, rolling windows of lagged counts.
 - Example: a lag window of 13 weeks provides 13 input features.

- No external regressors.

Modelling Details:

- Random Forest Regressor trained to predict the count 4 weeks ahead.
- Learns nonlinear relationships in lagged counts.

Output:

- Predicted counts for each hex in the test set.
- Compared against actual counts.

3. LSTM Regressor (*LSTM_PredModel.py*)

Approach:

- Single model trained across all hexes.

Input:

- Rolling windows of lagged weekly counts.
- Data normalized for stability.

Modelling Details:

- LSTM neural network learns temporal dependencies.
- The last hidden state feeds a dense layer to produce predictions.

Output:

- Predicted counts 4 weeks ahead.
- Predictions rescaled back to the original count scale.

4. LSTM with Seasonal Features (*LSTM_seasonal.py*)

Approach:

- Same as the standard LSTM but includes:
 - Sine and cosine encodings of week numbers to capture seasonality.

Model Performance Summary

Model	Input Window (weeks)	MAE	RMSE	R ²	MAPE
SARIMAX	49	2.3682	4.5821	0.6033	63.33%
Random Forest Regression	4	3.1654	6.3667	0.6350	77.13%
	13	2.8726	5.5582	0.7009	71.70%
	26	2.7900	5.5177	0.7497	62.48%
	48	2.7646	6.1100	0.6074	63.59%
LSTM	4	3.1109	5.9851	0.6191	74.20%
	13	2.2572	5.2863	0.7028	62.60%
	26	2.2050	5.1519	0.7178	59.38%
	48	3.6031	6.8040	0.5077	113.65%
LSTM + Seasonal Features	4	2.6511	5.6269	0.6633	64.00%
	13	2.9466	5.4652	0.6824	73.84%
	26	2.9359	5.8973	0.6302	66.78%
	48	3.5432	5.6061	0.6658	92.54%

Best Model:

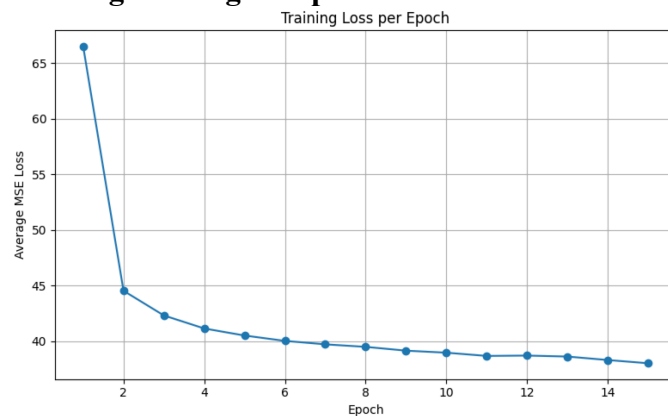
- **LSTM (26-week historical window)**
 - **MAE:** 2.2050
 - **RMSE:** 5.1519
 - **R²:** 0.7178
 - **MAPE:** 59.38%

Model Interpretation

LSTM Model Configuration:

- Hidden size: 64
- Layers: 2

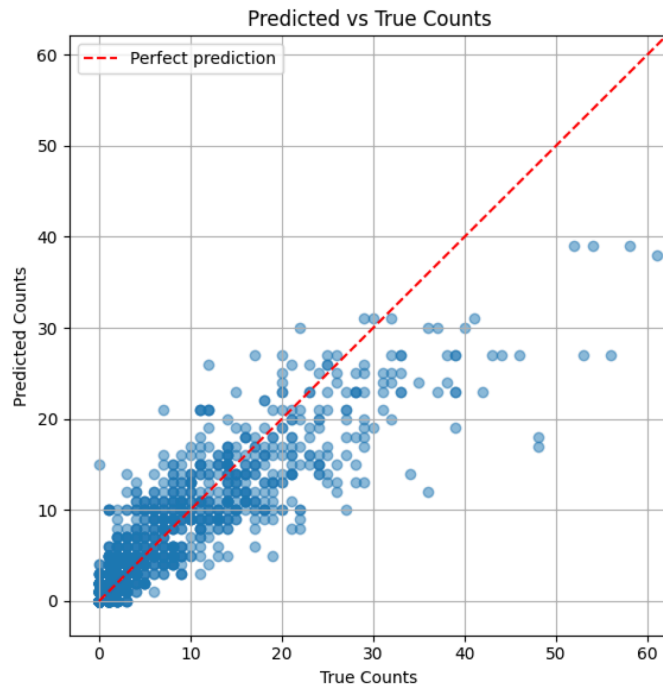
Training convergence plot:



Metrics:

- **MAE:** On average, predictions were ~2 counts off.
- **RMSE:** Errors typically ~5 counts.
- **R²:** ~72% of variance explained.
- **MAPE:** Errors ~59% relative to true counts (larger relative errors for small counts).

Graphical representation of results:



Interpretation:

The LSTM captured most trends ($R^2 \sim 0.72$) but still showed proportionally high errors for smaller counts, indicating room for improvement.

Final Predictions

- After identifying the best model, it was retrained on the **full dataset** (no train/test split).
- Predicted counts for each hex for the next 4 weeks.
- Results saved to [predictions.csv](#).

Excerpt from final model predictions.

hex_index	week_1_pred	week_2_pred	week_3_pred	week_4_pred
88ad361229ffff	16.0	15.0	15.0	15.0
88ad36122bffff	40.0	40.0	41.0	40.0
88ad36122dffff	14.0	14.0	14.0	15.0
88ad361231ffff	25.0	26.0	27.0	27.0
88ad361233ffff	8.0	8.0	8.0	8.0
88ad361235ffff	39.0	38.0	38.0	37.0
88ad361237ffff	21.0	22.0	22.0	22.0
88ad361239ffff	10.0	10.0	10.0	10.0
88ad36123bffff	1.0	1.0	1.0	1.0
88ad36123dffff	25.0	24.0	25.0	25.0
88ad361241ffff	13.0	13.0	13.0	13.0